



Learning Curve-Informed RL for Efficient Hyperparameter Optimization

Han-Syuan Liao¹ Cheng-An Tsai¹ Yi-Xiang Yu¹

¹Department of Computer Science, National Yang Ming Chiao Tung University

Motivation: tune like a human expert

Hyperparameter optimization (HPO) is costly — every candidate configuration needs a (partial) training run. RL-based HPO such as **Hyp-RL** learns a sequential search policy, but treats each evaluation as a *black-box scalar*, throwing away the **learning curve** produced during training.

- Human experts read the *geometry* of the curve — is the loss still dropping fast, decelerating, or plateauing? — not just the final number.
- These derivative signals have a *task-independent* physical meaning, so they are a natural substrate for **transfer across datasets**.

Our idea: two cheap augmentations to RL-HPO

- (i) **Derivative-informed state**. Append first- & second-order derivatives of the observed learning curve ($c'(k)$, $c''(k)$) to the agent's state — structured signal at **zero extra evaluation cost**.
- (ii) **Cross-dataset transfer**. Meta-train one shared LSTM policy across many datasets so it generalizes to *unseen* tasks without restarting from scratch.

Question: *when* do learning-curve derivatives actually help?

Framework: a learning curve-informed RL loop

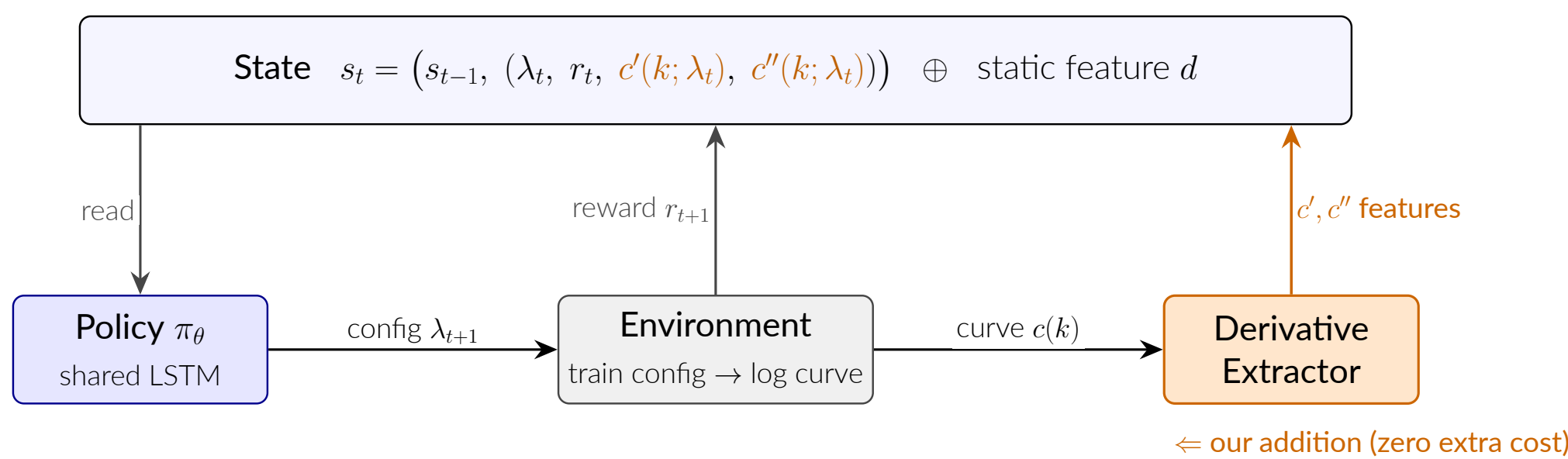
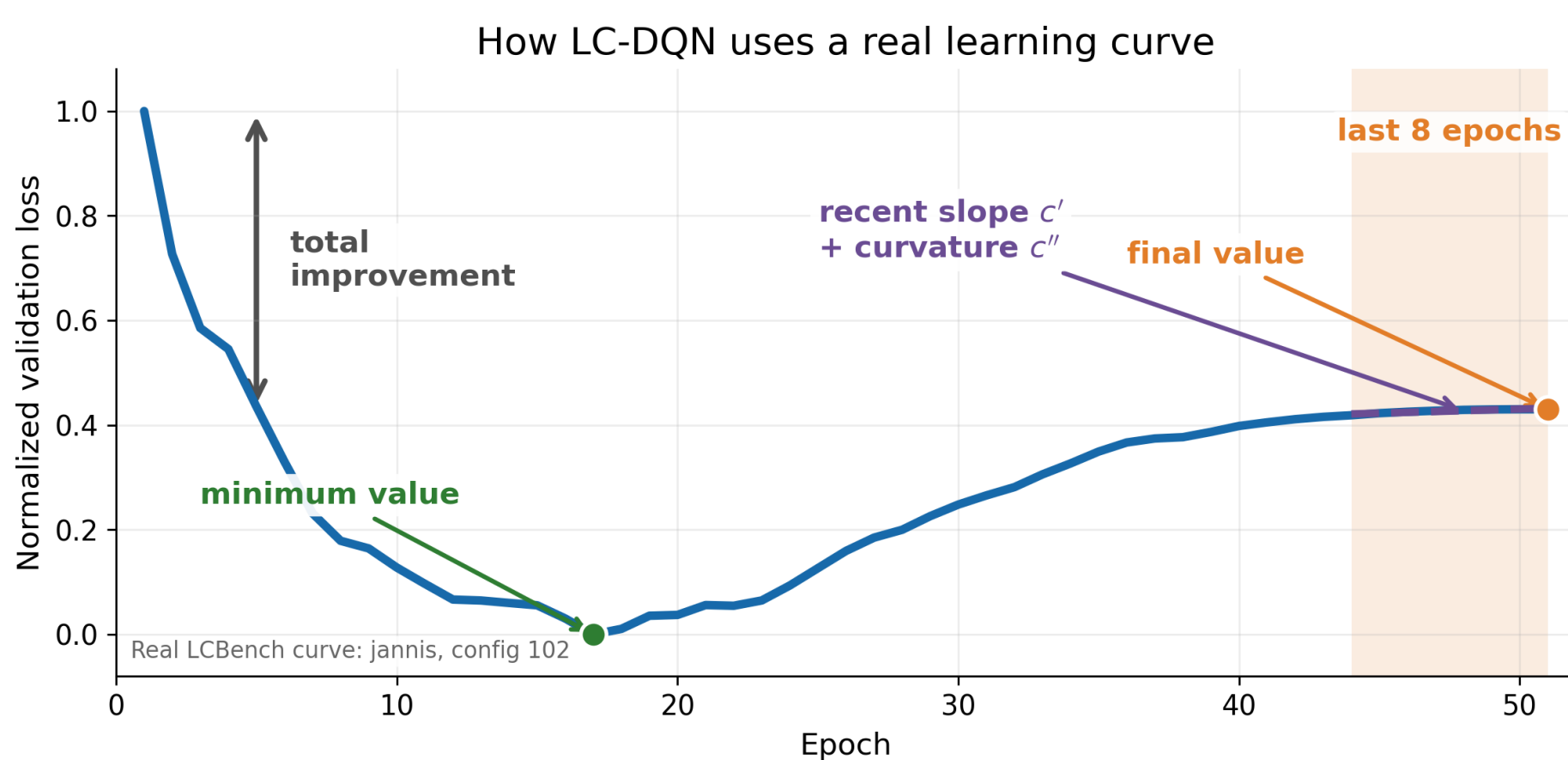


Figure 1. Learning curve-informed RL loop. The policy reads the state and picks a configuration λ_{t+1} ; the environment trains it and logs the learning curve $c(k)$; the **Derivative Extractor** turns that curve into first/second-order features c' , c'' appended to the next state — structured signal at *zero extra evaluation cost*.

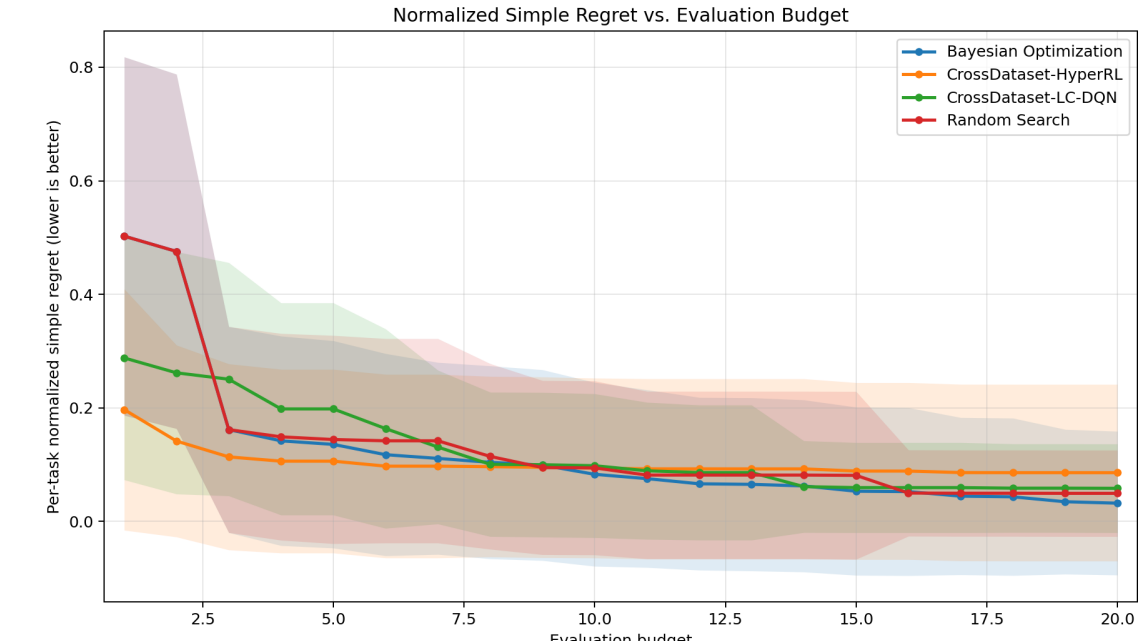
State $\rightarrow$ Policy $\rightarrow$ Environment $\rightarrow$ Derivative Extractor $\rightarrow$ State

We formulate HPO as an MDP: action = choosing a configuration λ , reward = negative generalization error $R(D, a) = -f(D, \lambda=a)$. An LSTM Q -network, primed by a 16-d dataset descriptor d , scores configurations. Each curve is summarized by **5 finite-difference features** (final value, min, slope, curvature, total improvement) over its last 8 epochs. The curve-free ablation is **HyperRL**; the full method is **LC-DQN**.

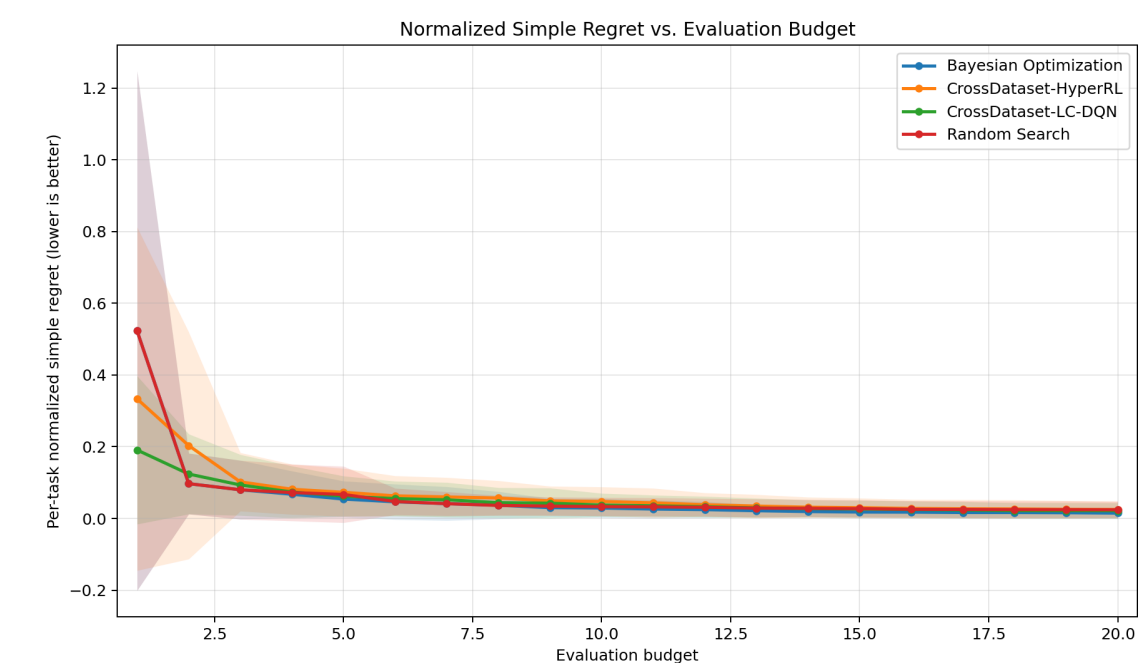


Key result: the same agent, two regimes

Identical agent and ablation (LC-DQN vs. curve-free HyperRL) on a homogeneous and a heterogeneous benchmark — the curve features help *only* when curve shapes vary.



(a) **LCBench (homogeneous):** LC-DQN (green) $\approx$ HyperRL (orange) — *no gain*.



(b) **Kaggle (heterogeneous):** LC-DQN (green) clearly below HyperRL (orange) — *consistent cold-start gain*.

Figure 2. Normalized simple regret vs. evaluation budget (lower is better); same two RL variants on both benchmarks.

Table 1. Average normalized simple regret on the 15 heterogeneous Kaggle Playground tasks (5 seeds; lower is better). The curve features are what push the RL agent below Random Search.

Method	Norm. regret $\downarrow$	Avg. rank
Bayesian Opt. (GP, asymptotic best)	0.0150	—
LC-DQN (ours, +curve features)	0.0217	2.33
Random Search	0.0243	—
HyperRL (curve-free ablation)	0.0251	2.87

The two variants differ *only* in the appended derivative features, so the Kaggle gap (0.0217 vs. 0.0251) is directly attributable to the learning-curve signal — transfer alone does not beat Random Search.

Conclusion: derivatives help when curve *shape* varies

- LCBench (homogeneous):** LC-DQN $\approx$ HyperRL — smooth, monotone curves are already well-predicted by the static descriptor + scalar reward, so derivatives add little.
- Kaggle (heterogeneous):** diverse curve geometries (steady improvers, early plateaus, overfitters) — derivatives supply discriminative cold-start signal, the *only* RL variant to beat Random Search.
- Cheap & zero-cost:** the derivative features are appended for free and pay off precisely when curve *shape* varies across configurations.
- Transfer + asymptotics:** the frozen cross-dataset policies start strong (transfer pays off at cold-start); Bayesian Optimization still wins asymptotically (0.0150).

Experimental setup

- LCBench:** 35 *homogeneous* OpenML tasks — same MLP family, 7-d search space, 50-epoch protocol $\Rightarrow$ stereotyped, near-monotone curves.
- Kaggle Playground Series:** 15 *heterogeneous* tabular tasks (regression & classification), diverse curve shapes $\Rightarrow$ the regime transfer is hard.
- Baselines: Random Search, Bayesian Optimization (Gaussian-process surrogate). Meta-policies frozen after training; evaluation budget = 20.

Future work

Surrogate-accelerated evaluation: replace full training runs with a multi-dataset surrogate, letting the agent exploit the curve signal across far more datasets at a fraction of the cost.